

CENTRO UNIVERSITÁRIO DA FUNDAÇÃO HERMÍNIO OMETTO

NÚCLEO DE ENGENHARIA



FUNDAÇÃO HERMÍNIO OMETTO

Proposta de um Sistema Multiagente Baseado em LLM para Orquestração de Módulos

Legados na RoboCup@Home

**A Multi-Agent LLM-Based Architecture for Orchestrating
Legacy Modules in RoboCup@Home**

Lucca Santos Cerasomma / 113653

ARARAS/SP, 2025

Resumo

A robótica de serviço, especialmente no contexto da competição RoboCup@Home Open Platform League (OPL), exige um nível crescente de autonomia, robustez e capacidade de planejamento. Este projeto parte de módulos de software legados e funcionais—especificamente um sistema de Processamento de Linguagem Natural (PLN) baseado em Large Language Models (LLMs) e um módulo de navegação simulado em ROS 2 e Gazebo. A integração existente baseia-se num pipeline reativo de passo único, inadequado para a decomposição de tarefas complexas ou para a recuperação de falhas. Este trabalho propõe o desenvolvimento de um novo módulo de raciocínio estruturado como um Sistema Multiagente (MAS) hierárquico. O módulo atua como um orquestrador central que trata os módulos legados como um "toolkit" de ferramentas invocáveis. A arquitetura proposta é um híbrido do estado da arte de padrões, empregando um Agente Supervisor para gerenciamento de estado e replanejamento, um Agente de Planejamento para decomposição de tarefas, e um Agente de Execução para a interface direta com os tópicos do ROS 2. Espera-se que este sistema permita ao robô executar autonomamente tarefas complexas, gerir contingências de forma dinâmica e satisfazer os exigentes requisitos de autonomia da OPL. A validação será realizada através de simulação, avaliando métricas de Taxa de Conclusão de Tarefa (TCR) e Taxa de Recuperação de Falha (FRR).

Palavras-chave: Sistema Multiagente. Large Language Models. Robótica de Serviço. RoboCup@Home. ROS 2. Planejamento de Tarefas.

Abstract

Service robotics, especially in the context of the RoboCup@Home Open Platform League (OPL) competition, demands an increasing level of autonomy, robustness, and planning capability. This project builds upon existing and functional legacy software modules—specifically a Natural Language Processing (NLP) system based on Large Language

Models (LLMs) and a navigation module simulated in ROS 2 and Gazebo. The existing integration relies on a single-step reactive pipeline, which is inadequate for complex task decomposition or fault recovery. This work proposes the development of a new reasoning module structured as a hierarchical Multi-Agent System (MAS). The module acts as a central orchestrator that treats legacy modules as an "invocable toolkit" of tools. The proposed architecture is a hybrid of state-of-the-art patterns, employing a Supervisor Agent for state management and replanning, a Planning Agent for task decomposition, and an Execution Agent for direct interfacing with ROS 2 topics. It is expected that this system will enable the robot to autonomously execute complex tasks, dynamically manage contingencies, and meet the demanding autonomy requirements of the OPL. Validation will be performed through simulation, evaluating metrics such as Task Completion Rate (TCR) and Failure Recovery Rate (FRR).

Key-words: Multi-Agent System. Large Language Models. Service Robotics. RoboCup@Home. ROS 2. Task Planning.

1 Introdução

A robótica de serviço em ambientes domésticos representa uma das fronteiras mais desafiadoras da engenharia de sistemas. O objetivo de criar agentes autônomos capazes de assistir humanos em tarefas diárias exige a integração sinérgica de múltiplos subsistemas complexos, incluindo Interação Humano-Robô (HRI), navegação e mapeamento em ambientes dinâmicos e não estruturados, e a manipulação precisa de objetos (**ROBOCUP@HOME, 2024**).

A competição internacional RoboCup@Home consolidou-se como o principal benchmark para o desenvolvimento e avaliação de robôs de serviço autônomos (**ROBOCUP@HOME, 2024**).

Dentro desta competição, a Open Platform League (OPL) é a categoria que oferece "o mais alto grau de liberdade" (**ROBOCUP@HOME, 2024**). A OPL permite que as equipes projetem seus próprios hardwares e, mais importante, incentiva a inovação em arquiteturas de software. Tarefas como o *Open Challenge* recompensam especificamente abordagens que demonstram pesquisa de

ponta e capacidade de generalização, indo além de testes roteirizados e exigindo um alto nível de raciocínio e integração de sistemas (**ROBOCUP@HOME, 2025**).

Este projeto não se inicia do zero. Ele se baseia em dois trabalhos de conclusão de curso (TCCs) anteriores, que estabelecem uma base de software robusta (**CASARIN; CAGNIN, 2025; 2025**). Esses trabalhos detalham o desenvolvimento e a validação de:

1. **Um Módulo de Processamento de Linguagem Natural (PLN) sofisticado:** Utiliza uma arquitetura híbrida de LLMs, empregando Gemma3 para classificação de comandos e DeepSeek para conversação natural (**CASARIN; CAGNIN, 2025**). O módulo é aprimorado com frameworks em Python como LangChain, para estruturação de fluxos de processamento, e Retrieval-Augmented Generation (RAG), para fornecer contexto específico do ambiente (ex.: regras da competição) e mitigar alucinações (**CASARIN; CAGNIN, 2025**).
2. **Sistemas de Interação por Voz:** O módulo de PLN está totalmente integrado com sistemas de Speech-to-Text (STT), como o Whisper da OpenAI, e Text-to-Speech (TTS), como Google Cloud TTS e Coqui.ai (**CASARIN; CAGNIN, 2025**).
3. **Uma Interface de Integração Definida:** O ponto mais crucial é que o trabalho legado já define uma interface de comunicação padronizada. O sistema utiliza ROS 2 Humble como middleware de mensageria e define o formato de saída de comandos como um dicionário JSON (**CASARIN; CAGNIN, 2025**). Este JSON é publicado em tópicos ROS 2 (ex.: *Action*, *Object*, *Room*) que um módulo de navegação consumidor, testado em simulação Gazebo, já está apto a interpretar (**CASARIN; CAGNIN, 2025**).

Apesar da funcionalidade de seus componentes individuais, a arquitetura de controle do sistema legado apresenta uma limitação fundamental que impede a verdadeira autonomia exigida pela OPL (**ROBOCUP@HOME, 2025**).

A arquitetura de controle descrita é um pipeline reativo de passo único.

O fluxo de dados é linear: uma entrada de usuário é recebida, um *Router* (baseado em LLM) a classifica e, em seguida, seleciona uma única ação — seja responder ao usuário ou publicar uma

única mensagem no ROS 2. Este Router não possui a capacidade de decompor uma tarefa complexa (CASARIN; CAGNIN, 2025).

Por exemplo, um comando da RoboCup@Home como “Vá à cozinha, pegue o refrigerante na geladeira e traga-o para o sofá da sala” não pode ser executado por este sistema. O Router legado não consegue decompor essa instrução numa sequência de subtarefas (ex.: *navigateTo(kitchen)*, *open(refrigerator)*, *findObject(soda)* etc.) (CASARIN; CAGNIN, 2025).

Além disso, o sistema legado carece de um mecanismo de gerenciamento de estado e recuperação de falhas. Se a primeira ação (*navigateTo(kitchen)*) falhar, o Router não tem como tomar conhecimento dessa falha e iniciar um replanejamento. Ele executou sua única ação e encerrou seu ciclo de processamento. Conclui-se que a arquitetura legada é funcional para comandos simples e discretos, mas falha estruturalmente em tarefas complexas que definem a autonomia na robótica de serviço (CASARIN; CAGNIN, 2025).

Dessa forma, o objetivo deste trabalho reside na substituição da arquitetura de roteamento reativo por um sistema multiagente (MAS) hierárquico, que atua como um “módulo de raciocínio” (ANGEL-FLORES et al., 2025), permitindo a orquestração e integração robusta dos módulos legados (CASARIN; CAGNIN, 2025).

Ao tratar os módulos de PLN e navegação existentes como um conjunto de “ferramentas” invocáveis (ROYCE et al., 2025), o novo sistema será capaz de decompor tarefas complexas, gerenciar o estado da missão e executar replanejamento dinâmico em caso de falhas, satisfazendo os requisitos de autonomia da OPL (ROBOCUP@HOME, 2025).

2 Hipótese

A hipótese deste projeto é que a adoção de uma arquitetura multiagente baseada em LLMs, operando como módulo de raciocínio central, aumentará significativamente a autonomia e a robustez do sistema robótico em comparação ao pipeline reativo legado, especialmente na execução de tarefas multi-etapa e na recuperação dinâmica de falhas durante operações domésticas simuladas na RoboCup@Home OPL.

3 Objetivos

3.1 Objetivo Principal

Desenvolver e validar um módulo de raciocínio multiagente baseado em LLMs capaz de orquestrar de forma autônoma os módulos legados de PLN, navegação e percepção, permitindo a execução robusta de tarefas complexas na plataforma RoboCup@Home OPL.

3.2 Objetivos Secundários

- Projetar uma arquitetura MAS hierárquica adequada ao fluxo de planejamento e execução de tarefas robóticas.
- Implementar três agentes especializados (Supervisor, Planejador e Executor) integrados ao ROS 2.
- Criar um conjunto de ferramentas ROS 2 padronizadas para invocação dos módulos legados.
- Desenvolver cenários de teste em Gazebo para validação funcional e avaliação de desempenho.
- Analisar as métricas TCR, FRR, PL e AL para quantificar o impacto da arquitetura proposta.

4 Fundamentação Teórica

Esta seção estabelece a base teórica para a integração proposta, detalhando os componentes legados que servem como ponto de partida e analisando os trabalhos relacionados que informam o design do novo módulo de raciocínio.

4.1 Módulo de PLN da Arquitetura Legada

O sistema legado fornece um módulo de PLN altamente capaz, cujas características definem os pontos de integração:

- **Componentes de Software:** O sistema utiliza uma arquitetura híbrida de LLMs, empregando Gemma3 para roteamento e DeepSeek para conversação.
- **Aprimoramento de Contexto:** O uso de LangChain para estruturar fluxos e RAG para

injetar conhecimento externo já estabelece um paradigma de "LLM aumentado".

- **Interface de Saída (ROS 2):** A arquitetura legada define explicitamente o formato de saída da classificação de comandos como um dicionário Python (serializável para JSON) com chaves predefinidas: *Action*, *Object*, *Location*, *Room*, *Direction*, *Person* e *Other*. O sistema publica essas informações em tópicos ROS 2. Esta interface ROS 2/JSON é o *contrato de API* fundamental que o novo módulo de raciocínio irá consumir.

4.2 Módulo de Navegação da Arquitetura Legada

Os TCCs confirmam o uso do *Robot Operating System* (ROS) e do simulador Gazebo para integração e teste.

Mais importante, os trabalhos da arquitetura legada revelam a existência de *tópicos de feedback*. Os módulos de navegação e percepção (visão) *já publicam seu estado* em tópicos ROS 2. O que facilita a integração dos supracitados módulos com o módulo de raciocínio proposto. Dessa forma, o novo módulo poderá se *inscrever* (subscribe) nesses tópicos de feedback. Isso permite a implementação de um *loop de controle fechado* (closed-loop control), um conceito fundamental para a arquitetura MARS e essencial para qualquer sistema de planejamento robusto.

4.3 O Paradigma Emergente: LLMs como Planejadores de Tarefas Robóticas

A literatura recente de pesquisa em robótica transcende o uso de LLMs de simples processadores de linguagem para motores centrais de Planejamento de Tarefas e Movimento (TAMP). LLMs demonstram uma capacidade robusta de decompor instruções de linguagem natural de alto nível diversas "primitivas de ação" de baixo nível. Estudos demonstram o uso de LLMs para gerar planos de ação sequenciais, raciocinar sobre restrições ambientais e até mesmo manter um modelo do estado do mundo.

4.4 Análise Comparativa de Arquiteturas MAS-LLM de Referência

Para projetar o módulo de raciocínio, três arquiteturas atuais do estado da arte foram analisadas para extrair seus padrões de design mais relevantes (Tabela 1).

1. **ROSA (Robot Operating System Agent):** esta arquitetura implementa um agente ReAct (*Reasoning and Acting*) que utiliza o LangChain para interagir diretamente com sistemas ROS 1 e ROS 2. Seu principal mecanismo é a definição de "ferramentas" (tools) Python que encapsulam comandos ros (ex: listar, ecoar ou publicar em tópicos) e invocam tarefas de navegação ou manipulação existentes. A contribuição de ROSA é o *mecanismo de integração* que conecta o LLM ao ROS.
2. **MALMM (Multi-Agent LLM for Manipulation):** este *framework* propõe uma arquitetura *hierárquica* de múltiplos agentes para mitigar alucinações em tarefas longas e complexas. Ele utiliza agentes especializados: um Agente Supervisor para gerenciar o fluxo da tarefa e coordenar outros agentes; um Agente Planner para gerar planos de alto nível; e um Agente Coder para traduzir esses planos em código executável. A contribuição de MALMM é o *padrão de design hierárquico (Supervisor-Planner)*, focado em gerenciamento de estado e *recuperação de falhas*.
3. **MARS (Multi-Agent Robotic System):** Esta arquitetura propõe um sistema colaborativo de *loop fechado (closed-loop)*
4. para assistência robótica. Utiliza agentes especializados, incluindo *Perception, Reasoning, Planning* e *Evaluation*. O Agente de Avaliação (*Evaluation*) verifica a viabilidade e a segurança dos planos gerados, solicitando revisões ao agente de planejamento. A contribuição de MARS é o *loop de feedback explícito* para validação de planos.

Tabela 1: Análise Comparativa de Arquiteturas MAS-LLM de Referência

Arquitetura	Padrão de Agente	Mecanismo de Integração	Gerenciamento de Falhas
ROSA ⁶	Agente Único ReAct (<i>Reasoning</i> + <i>Acting</i>)	Chamada de Ferramenta LangChain (invoca APIs ROS)	Limitado (depende do ReAct e da ferramenta)
MALMM ⁷	Hierárquico	Geração de Código Python	Explícito

	(Supervisor-Trabalhador)	(executado por um executor)	(Supervisor detecta falha e solicita replanejamento ao Planner)
MARS ⁸	Loop Fechado Colaborativo	Comunicação interna de agentes (ex: Agente de Percepção para Agente de Raciocínio)	Explícito (Agente de Avaliação revisa plano <i>antes</i> da execução)

5 Proposta da nova Arquitetura do Sistema

A análise da arquitetura legada (**CASARIN; CAGNIN, 2025**) e dos trabalhos de referência (Tabela 1) leva à proposta de uma nova arquitetura de sistema multiagente. Este sistema atuará como o módulo de raciocínio central, orquestrando os módulos legados (**CASARIN; CAGNIN, 2025**).

5.1 Visão Geral da Arquitetura Híbrida

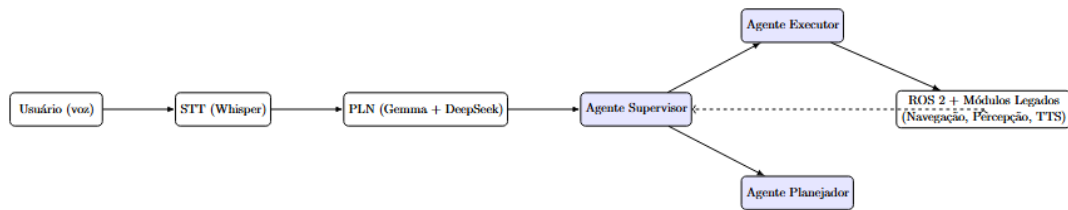
O design da arquitetura híbrida combina as vantagens das três arquiteturas de referência para resolver a lacuna do sistema legado:

1. **Hierarquia do MALMM (GHAFAROLLAHI et al., 2025)**: Adota-se a estrutura de um Agente Supervisor para gerenciamento de estado e um Agente de Planejamento para decomposição de tarefas.
2. **Mecanismo de Execução do ROSA (ROYCE et al., 2025)**: Substitui-se o Agente Coder do MALMM (que gera Python) por um Agente de Execução. Este agente utiliza o mecanismo de "ferramentas" LangChain do ROSA (**ROYCE et al., 2025**) para invocar os módulos ROS 2 legados (**CASARIN; CAGNIN, 2025**) de forma segura e controlada.

3. **Loop de Feedback do MARS (GAO; ZHONG, 2025):** O Agente Supervisor assume também o papel do Agente de Avaliação. Ele se inscreve ativamente nos tópicos de feedback do ROS 2 (CASARIN; CAGNIN, 2025) para monitorar a execução em tempo real, permitindo-lhe detectar falhas e disparar o replanejamento.

Figure 1: Arquitetura geral do sistema multiagente proposto, mostrando a integração entre PLN, agentes e ROS 2.

Figura 1 - Arquitetura geral do sistema multiagente proposto, mostrando a integração entre PLN, agentes e ROS 2.



Fonte: O autor (2025).

5.2 Definição dos Agentes Especializados (O Módulo de Raciocínio)

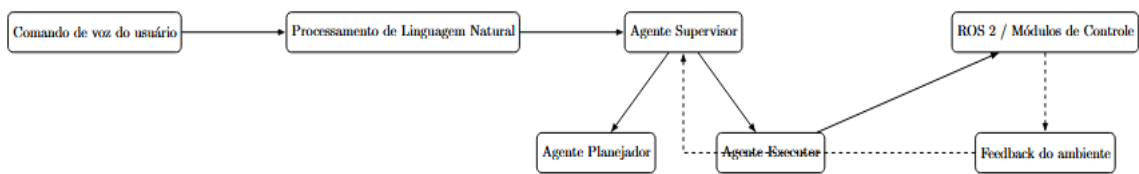
A nova arquitetura é composta por três agentes LLM especializados, cada um com uma responsabilidade única, conforme detalhado na Tabela 2.

1. **Agente Supervisor:** É o cérebro do robô. Mantém o estado da missão, recebe a intenção do usuário (CASARIN; CAGNIN, 2025), solicita planos ao planejador, despacha etapas de execução únicas para o agente de execução, e monitora os tópicos de feedback do ROS 2 para gerenciamento de falhas.
2. **Agente de Planejamento:** Especialista em decomposição de tarefas (LIU, 2025). Recebe um objetivo de alto nível do supervisor (ex: "buscar refrigerante") e a decompõe em uma

sequência de etapas de execução (chamadas de ferramentas).

- 3. **Agente de Execução:** Um agente LangChain ReAct (ROYCE et al., 2025) focado exclusivamente na invocação de ferramentas. Recebe um comando de etapa única do supervisor (ex: "CALL \$tool_navigateTo(room='kitchen')\$") e o executa.

Figura 2: Fluxo de dados entre os módulos do sistema híbrido proposto.



Fonte: O autor (2025).

Tabela 2: Definição dos papéis dos Agentes

Agente	Responsabilidade Primária	Inspiração das arquiteturas do estado da arte
Supervisor	Gerenciamento de Estado Feedback e Falhas. Orquestração.	MALMM Supervisor, MARS Evaluation

Planejamento	Decomposição de Tarefas de Alto Nível. Replanejamento.	MALMM Planner, MARS Planner
Execução	Invocação de Ferramentas (ROS 2). Interface de Ação.	ROSA Agent

5.3 A Camada de Integração: Ferramentas ROS 2

Esta é a ponte de software que conecta o novo módulo (Seção 3.2) aos módulos Legados (CASARIN; CAGNIN, 2025).

Consiste em funções Python (ROYCE et al., 2025) disponibilizadas ao Agente de Execução via LangChain (ROYCE et al., 2025).

Cada ferramenta é um wrapper que: (1) aceita argumentos Python simples e (2) formata e publica a mensagem JSON (CASARIN; CAGNIN, 2025) no tópico ROS 2 correto. A Tabela 3 detalha este contrato de API.

Tabela 3: Mapeamento de Ferramentas (Tools) para Tópicos ROS 2 Legados

Nome da Ferramenta (Tool)	Parâmetros Python	Módulo Legado Alvo	Estrutura da Mensagem JSON
\$tool_navigateTo\$	room: str, location: str	Módulo de Navegação	{"Action": "go_to", "Room": room, "Location": location,...}
\$tool_findPerson\$	person_name:	Módulo de	{"Action": "find", "Person":

n_Search\$	str	Percepção	person_name,...}
\$tool_findObject_Search\$	object_name: str	Módulo de Percepção	{"Action": "find", "Object": object_name,...}
\$tool_speakToUser\$	message: str	Módulo de Interação (TTS)	{"text": message}
\$tool_queryKnowledgeBase\$	query: str	Módulo de PLN (RAG)	Retorna str (ex: peso do objeto ³)
\$tool_getNavigationStatus\$	N/A	Módulo de Navegação	Retorna str ("success", "failed", "in_progress")

6. Materiais e Métodos

6.1 Materiais

- Desktop ou workstation com placa de vídeo compatível com execução local de LLMs (ex.: RTX 3060 ou superior).
- Sistema operacional Linux Ubuntu 22.04 ou equivalente.
- ROS 2 Humble Hawksbill instalado.
- Simulador Gazebo Garden/Ignition.
- Python 3.10+ e pacotes necessários (rclpy, LangChain, OpenAI-whisper, frameworks de LLM).
- Conjunto de módulos legados de PLN (Gemma3/DeepSeek), STT/TTS, percepção e navegação previamente implementados.
- Documentação da RoboCup@Home OPL (rulebook).

6.2 Métodos

- Análise da arquitetura legada e mapeamento das interfaces ROS 2 existentes.
- Construção da arquitetura multi-agente inspirada nas referências MALMM, ROSA e MARS.
- Implementação de ferramentas Python que encapsulam comandos ROS 2.
- Desenvolvimento dos três agentes LLM seguindo o padrão Supervisor → Planner → Executor.
- Testes incrementais: unitários, de integração e cenários simulados completos.
- Coleta das métricas definidas e análise quantitativa dos resultados.

Este projeto não envolve seres humanos nem animais, portanto dispensa aprovação do CEP e CEUA, conforme normas vigentes."

7 Metodologia de Integração e Fluxos de Operação

Esta seção detalha como a nova arquitetura multi-agente se integra com os módulos legados existentes (**CASARIN; CAGNIN, 2025**), substituindo o roteador reativo original por um fluxo de trabalho mais robusto e inteligente. Através de estudos de caso, será demonstrado como o novo módulo lida com tarefas complexas e gerencia contingências, superando as limitações do sistema legado.

7.1 Substituição do Roteador Legado

Será necessária a refatoração do fluxo de entrada do sistema legado (**CASARIN; CAGNIN, 2025**). O Router monolítico será descontinuado.

Novo Fluxo de Entrada:

1. Uma entrada de voz do usuário é processada pelo Whisper (STT) (**CASARIN; CAGNIN, 2025**) e convertida em texto
2. O texto é enviado ao Módulo PLN (Gemma3) (**CASARIN; CAGNIN, 2025**), cuja responsabilidade agora é apenas traduzir a linguagem natural ambígua em uma intenção

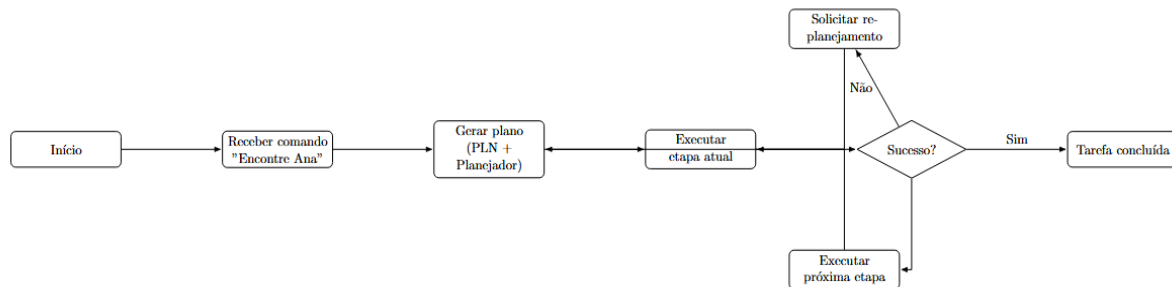
estruturada (o JSON de classificação (**CASARIN; CAGNIN, 2025**)).

3. Esta intenção JSON é então enviada ao Agente Supervisor, que inicia o ciclo de raciocínio.

Esta abordagem adere ao “Princípio da Responsabilidade Única”, tornando o módulo PLN legado mais simples e robusto.

7.2 Estudo de Caso 1: Tarefa Complexa (Busca Multi-Cômodos)

Figura 3: Fluxograma do caso de uso “Encontre Ana”



Fonte: O autor (2025).

Para ilustrar o valor da nova arquitetura, a Tabela 4 compara a execução da tarefa "Encontre Ana" (**CASARIN; CAGNIN, 2025**).

Tabela 4: Comparação de Fluxo de Tarefa para "Encontre Ana"

Fluxo Legado	Fluxo Proposto (MAS-Cogito)
1. Usuário -> PLN (Router) -> "Classificação e Ações".	1. Usuário -> PLN -> Intenção JSON -> Supervisor.
2. O módulo "Classificação" invoca uma <i>única e complexa</i> ferramenta <code>find_person('Ana')</code> .	2. Supervisor -> Planejador: "Planeje 'encontrar Ana'".
3. Esta ferramenta <code>find_person</code> contém <i>toda a lógica de negócios</i> (buscar cômodos, navegar, procurar, repetir).	3. Planejador (raciocina): "Ana é desconhecida. Devo buscar em todos os cômodos. (via <code>\$tool_queryKnowledgeBase\$</code>) Cômodos = ['bedroom', 'kitchen',...]".
4. É um <i>script</i> monolítico, difícil de manter e propenso a falhas catastróficas.	4. Planejador -> Supervisor (Plano): ``
	5. Supervisor executa Etapa 1: -> Executor -> <code>\$tool_navigateTo('bedroom')\$</code> .
	6. Supervisor (aguarda feedback): Ouve <code>\$TCC/navigation_status\$</code> . (módulo legado)
	7. Módulo Navegação publica: {"status": "success"}.
	8. Supervisor recebe feedback, executa Etapa 2:

	->Executor -> \$tool_findPerson_in_Room('Ana')\$.
	9. Módulo Percepção publica: {"status": "not_found"}.
	10. Supervisor recebe feedback, executa Etapa 3: -> Executor -> \$tool_navigateTo('kitchen')\$.
	11.... (Loop continua até que)...
	12. Módulo Percepção publica: {"status": "found", "person": "Ana"}.
	13. Supervisor recebe feedback: Plano concluído com sucesso.

7.3 Estudo de Caso 2: Gerenciamento de Contingências e Replanejamento

Cenário: O robô falha na Etapa 10 (navegar para a cozinha) porque a porta está bloqueada.

- **Comportamento Legado (CASARIN; CAGNIN, 2025):** O script monolítico find_person falharia. O robô pararia ou retornaria ao usuário, incapaz de concluir a missão, pois a lógica de recuperação para "caminho bloqueado" não foi explicitamente codificada pelo programador.
- **Comportamento nova arquitetura (Recuperação de Falha):**
 1. (Continua da Etapa 9 acima)
 2. Supervisor (executa Etapa 3): -> Executor -> \$tool_navigateTo('kitchen')\$.
 3. Módulo Nav (após detecção de obstáculo): Publica {"status": "failed", "reason": "path_blocked"} no tópico \$TCC/navigation_status\$.

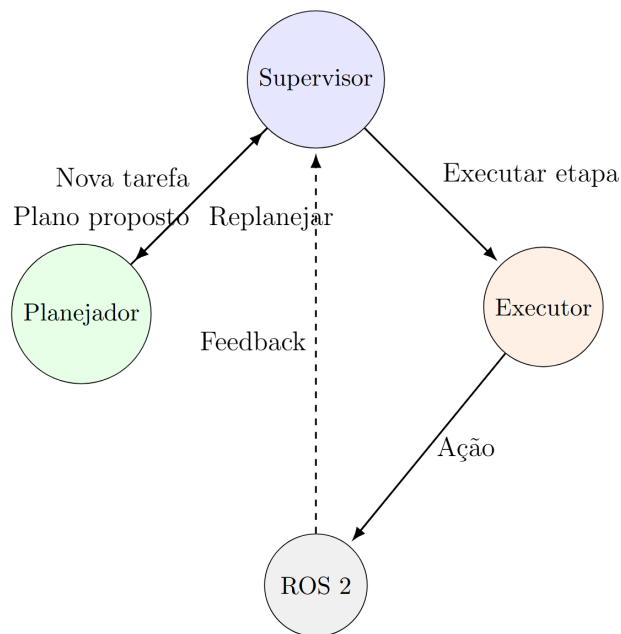
4. Supervisor (recebe feedback): "Falha na Etapa 3. Missão NÃO abortada."
5. Supervisor -> Planejador (Planejamento): "Recebi o plano [...list...]. A Etapa 3 falhou com 'path_blocked'. Gere um novo plano para 'encontrar Ana', excluindo a 'kitchen' ou tentando-a por último."
6. Planejador (raciocina): "OK. O caminho para 'kitchen' está bloqueado. Continuarei com o próximo cômodo da lista original."
7. Planejador -> Supervisor (Novo Plano): ``.
8. Supervisor (adota novo plano): Continua a execução da nova Etapa 1.

Esta arquitetura, inspirada no MALMM (**GHAFAROLLAHI et al., 2025**), transfere a responsabilidade da recuperação de falhas do programador (que teria que codificar manualmente cada cenário) para o Agente de Planejamento, que pode raciocinar dinamicamente sobre a falha e gerar um novo plano.

8 Resultados Esperados e Plano de Validação

Esta seção descreve os resultados esperados do projeto e o plano de validação detalhado para avaliar o desempenho do sistema multi-agente proposto. Serão apresentados os entregáveis, as contribuições esperadas e as métricas quantitativas que serão utilizadas para medir o sucesso da nova arquitetura.

Figura 4: Gerenciamento do planejamento



Fonte: O autor (2025).

8.1 Entregáveis do Projeto

1. Um pacote ROS 2 implementado em Python, contendo os três agentes **(ROYCE et al., 2025)**.
2. Arquivos de configuração e prompts **(YILDIRIM et al., 2025)** para cada agente LLM, definindo seus papéis, capacidades e o Toolkit de Ferramentas.
3. Um conjunto de cenários de teste de simulação (Gazebo) **(CASARIN; CAGNIN, 2025)**

para validar tarefas complexas (Estudo de Caso 1) e cenários de falha (Estudo de Caso 2).

4. Documentação técnica e um relatório final emulando o formato deste documento (CASARIN; CAGNIN, 2025).

8.2 Desempenho e Contribuição Esperados

Espera-se que o sistema resulte em um robô capaz de executar com sucesso tarefas multi-etapa de alto nível (ex: "Sirva uma bebida", "Limpe a sala") e de se adaptar a pelo menos uma falha inesperada por tarefa (ex: caminho bloqueado, objeto não encontrado).

A contribuição científica será a validação de uma arquitetura MAS-LLM híbrida (MALMM+ROSA) (ROYCE et al., 2025) como um método eficaz para integrar e orquestrar módulos robóticos ROS 2 legados, fornecendo um blueprint prático para equipes da RoboCup@Home (ROBOCUP@HOME, 2024) e pesquisadores de robótica de serviço.

8.3 Métricas de Avaliação

O sucesso do módulo de raciocínio será avaliado quantitativamente:

- **Taxa de Conclusão de Tarefa (TCR):** (Métrica primária). A percentagem de tarefas multi-etapa complexas (ex: "buscar e entregar") que o robô conclui com sucesso, sem intervenção humana.
- **Taxa de Recuperação de Falha (FRR):** (Métrica de robustez). A percentagem de falhas de execução introduzidas (ex: falha de navegação) das quais o Supervisor consegue se recuperar e concluir a missão através do replanejamento.
- **Latência de Planejamento (PL):** O tempo (em segundos) que o Planejador leva para decompor uma nova tarefa.
- **Latência de Ação (AL):** O tempo (em segundos) desde a recepção de um feedback de sucesso pelo Supervisor até o envio do próximo comando de etapa para o Executor, medindo a reatividade do sistema.

9 Cronograma

Tabela 5: Cronograma de Execução das Atividades

	Mês											
Atividade	J a n	F e v	M a r	A b r	M a i	J u n	J u l	A g o	S e t	O t	N o v	D e z
Refatoração da Interface Legada ³	X	X										
<i>Exposição dos tópicos de feedback (status)</i>		X										
Implementação do Toolkit e Agente Executor			X	X	X							
<i>Codificação do Toolkit de Ferramentas (Tabela 3)</i>			X	X								
<i>Implementação e teste do Agente executor</i>					X							
Implementação dos Agentes de Raciocínio						X	X	X				
<i>Engenharia de prompt do Agente planejador</i>						X	X					
<i>Implementação do loop de estado do Agente Supervisor</i>						X	X	X				
Testes de Integração e Validação (Gazebo)									X	X		
<i>Execução dos Estudos de Caso (Seção 4.2, 4.3)</i>									X	X		

<i>Coleta de dados para métricas (TCR, FRR)</i>											X		
<i>Análise de resultados e redação do relatório</i>												X	X
<i>Submissão de artigo científico</i>													X

10 Orçamento

As ferramentas são todas Open-Source, além disso, o projeto pode ser plenamente desenvolvido com um Desktop com placa de vídeo 3D.

Referências

AHMED, M. et al. A Survey on the Integration of Large Language Models (LLMs) into Autonomous Robotic Systems. MDPI, 2025.

ANGEL-FLORES, J. et al. Agentic Workflows Enhance LLM-based Reasoning in Semantic-Aware Robotic Task Planning. MDPI, 2025.

BISK, Y. et al. Collaborative Workflows: Specialized agents coordinate tasks... MDPI, 2025.

CASARIN, V. F.; CAGNIN, R. L. Desenvolvimento de um módulo para processamento de linguagem natural para robôs da categoria RoboCup@Home. Fundação Hermínio Ometto, 2025. (TCC_PARCIAL.pdf).

CASARIN, V. F.; CAGNIN, R. L. Desenvolvimento de um módulo para processamento de linguagem natural para robôs da categoria RoboCup@Home. Fundação Hermínio Ometto, 2025. (TCC (1).pdf).

CHEN, J. et al. Multi-Agent Systems for Robotic Autonomy with LLMs. arXiv preprint arXiv:2505.05762v1, 2025. (lido)

GAO, R.; ZHONG, P. MARS: Multi-Agent Robotic System with Multimodal Large Language Models for Assistive Intelligence. arXiv preprint arXiv:2511.01594v1, 2025.

GHAFAROLLAHI, M. et al. MALMM: Multi-Agent Large Language Model for Manipulation. arXiv preprint, 2025. (PDF Snippet).

GHAFAROLLAHI, M. et al. MALMM: Multi-Agent Large Language Model for Manipulation. arXiv preprint, 2025.

HlfShell. LLM Task Planners: Action Primitives and Execution. 2025. Disponível em: <https://hlfshell.ai/posts/llm-task-planner/>. Acesso em: 10 nov. 2025.

LI, M. et al. An AI Agent Framework Based on Large Language Models for Home Assistance Robots. NCBI, 2025. (quero ler de proximo)

LIU, P. Leveraging Large Language Models (LLMs) for Task and Motion Planning (TAMP) in Robotics. arXiv preprint arXiv:2510.16767v1, 2025. (lido)

LIU, Z. et al. LightPlanner: A Lightweight LLM-based Task Planning Framework for Resource-Constrained Robots. arXiv preprint arXiv:2503.08508v1, 2025.

Centro Universitário da Fundação Herminio Ometto, 2025. (Projeto Marcela.pdf).

NASA JPL. ROSA GitHub Repository. 2025. Disponível em: <https://github.com/nasa-jpl/rosa>. Acesso em: 10 nov. 2025.

NASA JPL. ROSA (ROS Agent) Discourse. Open Robotics Discourse, 2024. Disponível em: <https://discourse.openrobotics.org/t/nasa-jpl-has-open-sourced-rosa-an-ai-agent-designed-to-inter>

act-with-ros-based-systems-using-natural-language-queries/39075.

Acesso em: 10 nov. 2025.

PACKT PUBLISHING. Integration of ROS 2 Robots with LLM Agents. 2025. Disponível em:

<https://subscription.packtpub.com/book/iot-and-hardware/9781836209010/17/ch17lv11sec25/>.

Acesso em: 10 nov. 2025.

PAN, H. et al. Recent Advances in Robot Navigation via Large Language Models: A Review.

ResearchGate, 2024.

ROBOCUP@HOME. About the RoboCup@Home league. 2024. Disponível em:

<https://athome.robocup.org/>. Acesso em: 10 nov. 2025.

ROBOCUP@HOME. Focus domains. 2024. Disponível em: <https://athome.robocup.org/>.

Acesso em: 10 nov. 2025.

ROBOCUP@HOME RULEBOOK (Draft 2026). 3.9 Open Challenge. 2025. Disponível em:

<https://robocupathome.github.io/RuleBook/rulebook/master.pdf>. Acesso em: 10 nov. 2025.

ROYCE, R. et al. Enabling Novel Mission Operations and Interactions with ROSA: The Robot Operating System Agent. arXiv preprint arXiv:2410.06472v1, 2025.

ROYCE, R. et al. ROSA: Tool Structure and Organization. arXiv preprint arXiv:2410.06472v1, 2025.

ROYCE, R. et al. ROSA: Reasoning and Acting (ReAct) agent. arXiv preprint arXiv:2410.06472v1, 2025.

ROYCE, R. et al. ROSA: Reasoning and Acting (ReAct) agent. arXiv preprint arXiv:2410.06472v2, 2025.

SINGH, I. Language Models Meet Classical Planners: To Make Smarter Robot Task Plans. USC RASC, 2025. Disponível em:

<https://rasc.usc.edu/blog/language-models-meet-classical-planners-to-make-smarter-robot-task-pl>

[ans/](#).

Acesso em: 10 nov. 2025.

SREERAM, P. Building a Home Service Robot... Part 3: Plan-and-Execute AI Agent (react_agent.py). Medium, 2025. Disponível em:

<https://medium.com/@psreeram/building-a-home-service-robot-a-learning-journey-part-3-f326af7c8e2d>.

Acesso em: 10 nov. 2025.

WANG, J. Review of LLM-based Agents for Navigation Tasks. arXiv preprint arXiv:2311.00530v4, 2025.

YILDIRIM, M. et al. Integration of GPT-4 and LLaMA 2 with a Mobile Robot for Offline NLP and Task Execution at the Edge. arXiv preprint arXiv:2405.17670v2, 2025.